

打刻（出退勤）機能の設計

ログインができるようになったことは、curlコマンドで確認できたと思います。

ここからはいよいよ本題の「勤怠管理（打刻機能）」に入ります。

前提

ここまでで、 `backend` フォルダ内は以下のフォルダ・ファイル構成にできていると思います。

```
./src/main/java
├── com
│   └── appspace
│       └── backend
│           ├── BackendApplication.java
│           ├── config
│           │   └── SecurityConfig.java
│           ├── controller
│           │   └── UserAccountController.java
│           ├── entity
│           │   └── UserAccount.java
│           ├── repository
│           │   └── UserAccountRepository.java
│           └── service
│               └── UserAccountService.java
```

追加ファイル

ここからは、「apply_attendance_records」テーブルに対応する機能を追加実装していきます。
今回は以下のファイルを追加しています。ファイルを開いて内容を確認してみてください。

```
/workspaces/flexible-attendance-app/backend/src/main/java/com/appspace/backend/entity/AttendanceRecord.java  
/workspaces/flexible-attendance-app/backend/src/main/java/com/appspace/backend/repository/AttendanceRecordRepository.java  
/workspaces/flexible-attendance-app/backend/src/main/java/com/appspace/backend/service/AttendanceRecordService.java  
/workspaces/flexible-attendance-app/backend/src/main/java/com/appspace/backend/controller/AttendanceController.java
```

これらのファイルは、出勤・退勤時刻や、どのユーザーの記録かを紐付ける構成になっています。

解説 1

`AttendanceRecordRepository.java` の `findFirstBy...OrderByIdDesc` 機能について解説します。

これは `Spring Data JPA` の機能で、これがあることで「IDの降順（＝最新）で、特定のユーザーのデータを1件だけ取ってくる」というSQLを自動生成してくれます。

解説 2

`AttendanceRecord.java` と `AttendanceRecordService.java` は、連携して以下のような「状態の管理」を行っています。

出勤（clockIn）の動き

1. `Repository` を通じて、その人の一番新しいデータをDBに見に行きます。
2. もし「出勤時間は入っているけど、退勤時間が空（null）」というデータがあれば、「この人はまだ仕事中だ」と判断し、二重出勤を防ぐためにエラーを投げます。
3. 問題なければ、新しい行（レコード）を作って保存します。

退勤（clockOut）の動き

1. `Repository` で最新データを取ってきます。
2. 今度は逆に、「出勤中」のデータがないと困るので、データがない場合や既に退勤済みの場合はエラーにします。
3. 「出勤中」データがあったら、**同じ行（レコード）の `clockOut` 列に今の時間を書き込み**ます。

これで、「1日の勤務が1行のデータとして完成する」という仕組みになります。

解説 3

ビジネスロジック（Service）が「判断」を行うのに対し、Controller（コントローラー）は「外の世界（フロントエンド）とバックエンドを繋ぐ窓口」の役割を果たします。

次に作成する、打刻専用の窓口である `AttendanceController.java` クラスには、これまでのコントローラーとは少し異なる「データの受け取り方」を盛り込んでいます。

① `@RequestParam` によるデータの受け取り

前回のユーザー登録では、データの塊（JSON）を受け取るために `@RequestBody` を使いました。

今回は、URLの末尾に `?accountId=xxx` という形でIDを添えて送る `@RequestParam` を採用しています。

- **使い分けのイメージ:**
- `@RequestBody` : 登録情報や設定など、項目が多いとき（重いデータ）。
- `@RequestParam` : 「誰が？」といった特定のIDを一つだけ指定するとき（軽いデータ）。

② try-catch によるエラーハンドリング

AttendanceRecordService で書いた `throw new RuntimeException(...)` をここでキャッチしています。

- もしServiceが「まだ退勤していません！」とエラーを出したら、Controllerがそれを拾って `400 Bad Request` というステータスをフロントエンドに返します。
- これにより、画面側で「まだ退勤していませんよ」といったアラートを出すきっかけを作ることができます。

③ 適切なエンドポイント（URL）設計

- `/api/attendance/clock-in` : 出勤ボタン用
 - `/api/attendance/clock-out` : 退勤ボタン用
- というように、直感的にわかりやすい名前を付けています。

SecurityConfig.java の更新

これらの新しい窓口も「ログイン前（テスト中）」に触れるように、許可リストへ追加しましょう。

```
// SecurityConfig.java の該当箇所を修正
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/api/users/register", "/api/users/login", "/api/attendance/**").permitAll()
    .anyRequest().authenticated()
)
```

打刻のテスト（curl）の手順

すべてを保存し、Spring Bootを再起動したら、以下の流れでテストしてみてください。

- **ユーザー登録**（H2がリセットされているため）
- **そのユーザーの `accountId` をメモする**（登録時のレスポンスに含まれます）
- **出勤打刻をする**

```
curl -X POST "http://localhost:8080/api/attendance/clock-in?accountId=【メモしたID】"
```

- **退勤打刻をする**

```
curl -X POST "http://localhost:8080/api/attendance/clock-out?accountId=【メモしたID】"
```

これで、出勤時間と退勤時間が1つのデータとして記録されるはずです！